If your goal is to become job-ready in Selenium Automation Testing, I recommend learning these topics **in this exact order**, because each topic builds on the previous one.

Prerequisites

Before starting, install:

- Java JDK 17 or later
- IntelliJ IDEA Community Edition
- Chrome Browser
- Maven (comes integrated with IntelliJ)
- Git (optional but recommended)

Project Structure You'll Build

```
SeleniumAutomation
│
├── src
│   ├── main
│   │     └── java
│   │           └── pages
│   │
│   └── test
│         ├── java
│         │      ├── tests
│         │      ├── stepdefinitions
│         │      └── runners
│         │
│         └── resources
│                └── features
│
├── pom.xml
```

STEP 1 — Create Maven Project in IntelliJ

1. Open IntelliJ

Click

```
New Project
```

Choose

```
Maven
```

Next

Name

```
SeleniumAutomation
```

Finish.

---

2. Open pom.xml

Replace with

XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>automation</groupId>
    <artifactId>SeleniumAutomation</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>

        <!-- Selenium -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.23.0</version>
        </dependency>

        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
```

```
            <scope>test</scope>
        </dependency>

        <!-- WebDriver Manager -->
        <dependency>
            <groupId>io.github.bonigarcia</groupId>
            <artifactId>webdrivermanager</artifactId>
            <version>5.9.2</version>
        </dependency>

        <!-- Cucumber -->
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-java</artifactId>
            <version>7.18.0</version>
        </dependency>

        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-testng</artifactId>
            <version>7.18.0</version>
        </dependency>

    </dependencies>

</project>
```

Click

```
Load Maven Changes
```

TOPIC 1 — Understanding BDD

What is BDD?

BDD = Behavior Driven Development

Instead of writing

```
Click Login
Enter Username
Click Submit
```

BDD says

```
Given User is on Login Page
```

```
When User enters Username and Password

Then User should see Dashboard
```

It focuses on **behavior**, making scenarios readable by developers, testers, and business stakeholders.

BDD Syntax

gherkin

```
Feature: Login

Scenario: Valid Login

Given User is on login page

When User enters username

And User enters password

Then Dashboard should open
```

Keyword meanings:

```
Feature
```

Entire functionality.

```
Scenario
```

Single test case.

```
Given
```

Precondition.

```
When
```

Action.

```
Then
```

Expected result.

```
And
```

Additional step.

Example

Feature:

```
Google Search
```

Scenario

```
Given Google is open

When user searches Selenium

Then results appear
```

TOPIC 2 — TestNG + Selenium

Create

```
tests

LoginTest.java
```

Java

```java
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;

public class LoginTest {

    WebDriver driver;

    @BeforeMethod
    public void setup() {

        WebDriverManager.chromedriver().setup();

        driver = new ChromeDriver();
```

```
        driver.manage().window().maximize();
    }

    @Test
    public void googleTest() {

        driver.get("https://google.com");

        System.out.println(driver.getTitle());

    }

    @AfterMethod
    public void teardown() {

        driver.quit();

    }
}
```

Run using the green ▶ button beside the class or method.

TestNG Annotations

```
Java
@BeforeSuite
```

Runs once before everything.

```
Java
@BeforeClass
```

Runs before class.

```
Java
@BeforeMethod
```

Runs before every test.

```
Java
@Test
```

Actual test.

```java
@AfterMethod
```

Cleanup.

TOPIC 3 — Refactoring in IntelliJ

Suppose

```java
driver.findElement(By.id("username")).sendKeys("admin");

driver.findElement(By.id("password")).sendKeys("admin123");

driver.findElement(By.id("login")).click();
```

appears in many test methods.

Extract Method

Select the repeated code.

Press

```
Ctrl + Alt + M
```

Name it

```
login()
```

IntelliJ creates

```java
public void login(){

driver.findElement(By.id("username")).sendKeys("admin");

driver.findElement(By.id("password")).sendKeys("admin123");

driver.findElement(By.id("login")).click();
```

Call it simply as

```
Java
login();
```

Rename Variable

Select variable

Press

```
Shift + F6
```

Rename

```
driver
```

to

```
webDriver
```

Every occurrence updates automatically.

Extract Constant

Instead of

```
Java
driver.get("https://google.com");
```

Select the URL.

Press

```
Ctrl + Alt + C
```

Result

```
Java
```

```java
private static final String URL="https://google.com";
```

TOPIC 4 — Page Object Model (POM)

Without POM:

```java
driver.findElement(By.id("username")).sendKeys("admin");

driver.findElement(By.id("password")).sendKeys("admin");

driver.findElement(By.id("login")).click();
```

If IDs change, every test must be updated.

Create LoginPage.java

```java
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    By username=By.id("username");

    By password=By.id("password");

    By login=By.id("login");

    public LoginPage(WebDriver driver){

        this.driver=driver;

    }

    public void login(String user,String pass){

        driver.findElement(username).sendKeys(user);

        driver.findElement(password).sendKeys(pass);
```

```
        driver.findElement(login).click();

    }

}
```

Use in Test

Java

```
LoginPage page=new LoginPage(driver);

page.login("admin","admin123");
```

Page Factory

Instead of

Java

```
By username=By.id("username");
```

use

Java

```
@FindBy(id="username")
WebElement username;
```

Constructor

Java

```
PageFactory.initElements(driver,this);
```

Complete example:

Java

```
@FindBy(id="username")
WebElement username;

@FindBy(id="password")
WebElement password;
```

```
@FindBy(id="login")
WebElement login;

public void login(String user,String pass){

username.sendKeys(user);

password.sendKeys(pass);

login.click();

}
```

Why POM?

Advantages:

- Less duplicate code
- Easy maintenance
- Better readability
- Reusable page classes
- Faster updates when locators change

TOPIC 5 — Cucumber Acceptance Tests

Create folder

```
resources/features
```

Create

```
login.feature
```

```gherkin
Feature: Login

Scenario: Valid Login

Given User opens login page

When User enters username and password
```

Step Definitions

Create

```
stepdefinitions

LoginSteps.java
```

Java

```java
import io.cucumber.java.en.*;

public class LoginSteps {

    @Given("User opens login page")
    public void openPage(){

        System.out.println("Open");

    }

    @When("User enters username and password")
    public void login(){

        System.out.println("Login");

    }

    @Then("User should login successfully")
    public void verify(){

        System.out.println("Success");

    }

}
```

Runner

Java

```java
import io.cucumber.testng.AbstractTestNGCucumberTests;
import io.cucumber.testng.CucumberOptions;
```

```
@CucumberOptions(

features="src/test/resources/features",

glue="stepdefinitions"

)

public class TestRunner extends AbstractTestNGCucumberTests {
}
```

Run the `TestRunner` class.

Integrating Selenium with Cucumber

Instead of just printing messages, use Selenium actions inside the step definitions:

Java

```
@Given("User opens login page")
public void openPage() {

    WebDriverManager.chromedriver().setup();

    driver = new ChromeDriver();

    driver.get("https://example.com/login");

}
```

Java

```
@When("User enters username and password")
public void login() {

    LoginPage page = new LoginPage(driver);

    page.login("admin","admin123");

}
```

Java

```
@Then("User should login successfully")
public void verify() {
```

```
    Assert.assertTrue(driver.getTitle().contains("Dashboard"));

    driver.quit();

}
```

Final Learning Flow

Follow this progression to build a strong foundation:

1. **Create a Maven project** and configure Selenium, TestNG, and WebDriverManager.
2. **Learn Selenium basics**: launch a browser, locate elements, interact with pages, and add assertions.
3. **Learn TestNG**: annotations, test lifecycle, grouping, priorities, parameters, and reports.
4. **Practice IntelliJ refactoring**: extract methods, rename symbols, extract constants, and organize code to reduce duplication.
5. **Implement the Page Object Model (POM)**: move locators and page-specific actions into dedicated page classes.
6. **Use Page Factory**: simplify element initialization with `@FindBy` and `PageFactory.initElements()`.
7. **Learn BDD with Cucumber**: write Gherkin feature files (`Feature`, `Scenario`, `Given`, `When`, `Then`).
8. **Connect Cucumber with Selenium and POM**: implement step definitions that call your page objects, and execute them using a Cucumber TestNG runner.

By the end of these steps, you'll have a maintainable automation framework where:

- **Selenium** performs browser interactions.
- **TestNG** manages test execution and reporting.
- **POM/Page Factory** organizes page-specific code.
- **Cucumber (BDD)** describes application behavior in readable scenarios that map to your Selenium tests.